Documentación API - Backend Clínica

Tabla de contenidos

1. Introducción
2. Autenticación
3. Roles y Autorización
4. Endpoints
    - Usuarios
    - Doctores
    - Citas
    - Especialidades
    - Horarios
    - Pacientes
    - Seguros de Salud
    - Ubicaciones
    - Administración
    - Emails
    - Estudios Médicos

Introducción

Esta es la documentación de la API REST del sistema de gestión de clínica. La API está construida con Express.js y Sequelize ORM, proporcionando endpoints para gestionar usuarios, citas, doctores, especialidades y más.

URL Base: `localhost`

Autenticación

JWT (JSON Web Tokens)

La API utiliza JWT para autenticación. El token debe ser enviado en el header `Authorization`:

```
Authorization: Bearer <token>
```

Login

Existen tres tipos de login:

1. **Usuarios (Pacientes):**
    - POST `/api/users/login`
    - Retorna un JWT con rol `patient`

2. **Doctores:**
   - POST `/api/doctors/login`
   - Retorna un JWT con rol `doctor`
3. **Administradores:**
   - POST `/api/admin/login`
   - Retorna un JWT con rol `admin`

Roles y Autorización

El sistema cuenta con los siguientes roles:

Rol	Descripción
`admin`	Acceso completo al sistema
`doctor`	Acceso a funcionalidades de doctores
`patient`	Acceso a funcionalidades de pacientes
`AdminOrPatient`	Requiere ser admin o paciente
`DoctorOrPatient`	Requiere ser doctor o paciente

Endpoints

Usuarios

1. Login de Usuario (Paciente)

```
POST /api/users/login
```

Descripción: Autentica un paciente y retorna un JWT

Body:

```
{
  "nationalId": "string",
  "password": "string"
}
```

Respuesta (200):

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": 1,
    "nationalId": "12345678",
    "firstName": "Juan",
    "lastName": "Pérez",
    "email": "juan@example.com"
  }
}
```

---

2. Obtener Usuario por dni

```
POST /api/users/nationalId
Authorization: Bearer <token>
```

Descripción: Obtiene información de un usuario por su dni

Roles Requeridos: `AdminOrPatient`

Body:

```
{
  "nationalId": "string"
}
```

Respuesta (200):

```
{
  "id": 1,
  "nationalId": "12345678",
  "firstName": "Juan",
  "lastName": "Pérez",
  "email": "juan@example.com"
}
```

---

3. Obtener Todos los Usuarios

```
GET /api/users/all
Authorization: Bearer <token>
```

Descripción: Obtiene la lista de todos los usuarios

Roles Requeridos: `Patient`

Respuesta (200):

```
[
  {
    "id": 1,
    "nationalId": "12345678",
    "firstName": "Juan",
    "lastName": "Pérez",
    "email": "juan@example.com"
  }
]
```

---

4. Crear Usuario (Registro)

```
POST /api/users
```

Descripción: Crea un nuevo usuario en el sistema

Body:

```
{
  "nationalId": "string",
  "firstName": "string",
  "lastName": "string",
  "email": "string",
  "password": "string",
  "phoneNumber": "string"
}
```

Respuesta (201):

```
{
  "id": 1,
  "nationalId": "12345678",
  "firstName": "Juan",
  "lastName": "Pérez",
  "email": "juan@example.com"
}
```

5. Actualizar Usuario

```
PUT /api/users
Authorization: Bearer <token>
```

Descripción: Actualiza la información de un usuario

Roles Requeridos: `AdminOrPatient`

Body:

```
{
  "nationalId": "string",
  "firstName": "string",
  "lastName": "string",
  "email": "string",
  "phoneNumber": "string"
}
```

Respuesta (200):

```
{
  "id": 1,
  "nationalId": "12345678",
  "firstName": "Juan",
  "lastName": "Pérez",
  "email": "juan@example.com"
}
```

---

6. Eliminar Usuario

```
DELETE /api/users/:nationalId
Authorization: Bearer <token>
```

Descripción: Elimina un usuario del sistema

Roles Requeridos: `AdminOrPatient`

Parámetros:

- `nationalId` - dni del usuario a eliminar

Respuesta (200):

```
{
  "message": "User deleted successfully"
}
```

Doctores

1. Login de Doctor

```
POST /api/doctors/login
```

Descripción: Autentica un doctor y retorna un JWT

Body:

```
{
  "nationalId": "string",
  "password": "string"
}
```

Respuesta (200):

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "doctor": {
    "id": 1,
    "nationalId": "12345678",
    "firstName": "Dr. Carlos",
    "lastName": "López",
    "email": "carlos@clinic.com"
  }
}
```

---

2. Obtener Doctores (Filtrados)

```
POST /api/doctors
Authorization: Bearer <token>
```

Descripción: Obtiene doctores filtrados por especialidad y ubicación

Roles Requeridos: `Patient`

Body:

```
{
  "locationId": 1,
  "specialtyId": 2
}
```

Respuesta (200):

```
[
  {
    "id": 1,
    "firstName": "Dr. Carlos",
    "lastName": "López",
    "specialtyId": 2,
    "locationId": 1
  }
]
```

---

3. Obtener Doctores Disponibles

```
POST /api/doctors/available
Authorization: Bearer <token>
```

Descripción: Obtiene doctores disponibles en una ubicación específica

Roles Requeridos: `Patient`

Body:

```
{
  "locationId": 1
}
```

Respuesta (200):

```
[
  {
```

```
    "id": 1,
    "firstName": "Dr. Carlos",
    "lastName": "López"
  }
]
```

---

4. Obtener Todos los Doctores (Admin)

```
POST /api/doctors/all
Authorization: Bearer <token>
```

Descripción: Obtiene la lista completa de todos los doctores

Roles Requeridos: `Admin`

Respuesta (200):

```
[
  {
    "id": 1,
    "nationalId": "12345678",
    "firstName": "Dr. Carlos",
    "lastName": "López",
    "email": "carlos@clinic.com"
  }
]
```

---

5. Obtener Doctor por ID

```
GET /api/doctors/:id
Authorization: Bearer <token>
```

Descripción: Obtiene la información detallada de un doctor

Roles Requeridos: `AdminOrPatient`

Parámetros:

- `id` - ID del doctor

Respuesta (200):

```
{
  "id": 1,
  "nationalId": "12345678",
  "firstName": "Dr. Carlos",
  "lastName": "López",
  "email": "carlos@clinic.com",
  "specialtyId": 2
}
```

---

6. Obtener Doctor por dni (Doctor)

```
GET /api/doctors/nationalId
Authorization: Bearer <token>
```

Descripción: Obtiene información del doctor autenticado

Roles Requeridos: `Doctor`

Body:

```
{
  "nationalId": "string"
}
```

Respuesta (200):

```
{
  "id": 1,
  "nationalId": "12345678",
  "firstName": "Dr. Carlos",
  "lastName": "López"
}
```

---

7. Crear Doctor (Admin)

```
POST /api/admin/doctors
Authorization: Bearer <token>
```

Descripción: Crea un nuevo doctor en el sistema

Roles Requeridos: `Admin`

Body:

```
{
  "nationalId": "string",
  "firstName": "string",
  "lastName": "string",
  "email": "string",
  "password": "string",
  "specialtyId": 1,
  "locationId": 1
}
```

Respuesta (201):

```
{
  "id": 1,
  "nationalId": "12345678",
  "firstName": "Dr. Carlos",
  "lastName": "López"
}
```

---

8. Actualizar Doctor (Admin)

```
PUT /api/admin/doctors/:id
Authorization: Bearer <token>
```

Descripción: Actualiza la información de un doctor

Roles Requeridos: `Admin`

Parámetros:

- `id` - ID del doctor

Body:

```
{
  "firstName": "string",
  "lastName": "string",
  "email": "string",
  "specialtyId": 1
}
```

Respuesta (200):

```
{
  "id": 1,
  "firstName": "Dr. Carlos",
  "lastName": "López"
}
```

---

9. Eliminar Doctor (Admin)

```
DELETE /api/admin/doctors/:id
Authorization: Bearer <token>
```

Descripción: Elimina un doctor del sistema

Roles Requeridos: `Admin`

Parámetros:

- `id` - ID del doctor

Respuesta (200):

```
{
  "message": "Doctor deleted successfully"
}
```

Citas

1. Obtener Citas del Paciente

```
POST /api/appointments/patient
Authorization: Bearer <token>
```

Descripción: Obtiene todas las citas de un paciente

Roles Requeridos: `Patient`

Body:

```
{
  "nationalId": "string"
}
```

Respuesta (200):

```
[
  {
    "id": 1,
    "dateTime": "2025-06-15T10:00:00",
    "doctorId": 1,
    "patientId": 1,
    "status": "confirmed"
  }
]
```

---

2. Obtener Citas del Doctor (Hoy)

```
POST /api/appointments/doctor/today
Authorization: Bearer <token>
```

Descripción: Obtiene las citas del doctor para el día actual

Roles Requeridos: `Doctor`

Respuesta (200):

```
[
  {
    "id": 1,
    "dateTime": "2025-06-15T10:00:00",
    "patientName": "Juan Pérez",
    "status": "confirmed"
  }
]
```

---

3. Obtener Historial de Citas del Doctor

```
POST /api/appointments/doctor/history
Authorization: Bearer <token>
```

Descripción: Obtiene el historial completo de citas del doctor

Roles Requeridos: `Doctor`

Respuesta (200):

```
[
  {
    "id": 1,
    "dateTime": "2025-06-15T10:00:00",
    "patientName": "Juan Pérez",
    "status": "completed"
  }
]
```

---

4. Obtener Citas del Doctor por Fecha

```
POST /api/appointments/doctor/date
Authorization: Bearer <token>
```

Descripción: Obtiene las citas del doctor para una fecha específica

Roles Requeridos: `Doctor`

Body:

```
{
  "date": "2025-06-15"
}
```

Respuesta (200):

```
[
  {
    "id": 1,
    "dateTime": "2025-06-15T10:00:00",
    "patientName": "Juan Pérez",
    "status": "confirmed"
  }
]
```

---

5. Crear Cita

```
POST /api/appointments
Authorization: Bearer <token>
```

Descripción: Crea una nueva cita

Roles Requeridos: `Patient`

Body:

```
{
  "patientId": 1,
  "doctorId": 1,
  "dateTime": "2025-06-15T10:00:00"
}
```

Respuesta (201):

```
{
  "id": 1,
  "patientId": 1,
  "doctorId": 1,
  "dateTime": "2025-06-15T10:00:00",
  "status": "pending"
}
```

---

6. Confirmar Cita

```
PUT /api/appointments/confirm
Authorization: Bearer <token>
```

Descripción: Confirma una cita pendiente

Roles Requeridos: `Patient`

Body:

```
{
  "appointmentId": 1
}
```

Respuesta (200):

```
{
  "id": 1,
  "status": "confirmed"
}
```

---

7. Cancelar Cita

```
PUT /api/appointments/cancel
Authorization: Bearer <token>
```

Descripción: Cancela una cita

Roles Requeridos: `Patient`

Body:

```
{
  "appointmentId": 1
}
```

Respuesta (200):

```
{
  "id": 1,
  "status": "cancelled"
}
```

---

8. Eliminar Cita

```
DELETE /api/appointments/:id
Authorization: Bearer <token>
```

Descripción: Elimina una cita del sistema

Parámetros:

- `id` - ID de la cita

Respuesta (200):

```
{
  "message": "Appointment deleted successfully"
}
```

Especialidades

1. Obtener Especialidades

```
POST /api/specialties
Authorization: Bearer <token>
```

Descripción: Obtiene especialidades disponibles en una ubicación

Roles Requeridos: `Patient`

Body:

```
{
  "locationId": 1
}
```

Respuesta (200):

```
[
  {
    "id": 1,
    "name": "Cardiología",
    "description": "Especialidad del corazón"
  }
]
```

---

2. Obtener Todas las Especialidades

```
POST /api/specialties/all
```

Descripción: Obtiene la lista completa de especialidades (sin autenticación)

Respuesta (200):

```
[
  {
    "id": 1,
    "name": "Cardiología"
  },
  {
    "id": 2,
    "name": "Dermatología"
  }
]
```

---

3. Obtener Especialidades Disponibles

```
POST /api/specialties/available
Authorization: Bearer <token>
```

Descripción: Obtiene especialidades con doctores disponibles en una ubicación

Roles Requeridos: `Patient`

Body:

```
{
  "locationId": 1
}
```

Respuesta (200):

```
[
  {
    "id": 1,
    "name": "Cardiología",
    "doctorsCount": 3
  }
]
```

---

4. Obtener Especialidad por ID

```
GET /api/specialties/:id
Authorization: Bearer <token>
```

Descripción: Obtiene la información de una especialidad específica

Roles Requeridos: `Patient`

Parámetros:

- `id` - ID de la especialidad

Respuesta (200):

```
{
  "id": 1,
  "name": "Cardiología",
  "description": "Especialidad del corazón"
}
```

---

5. Crear Especialidad (Admin)

```
POST /api/admin/specialties
Authorization: Bearer <token>
```

Descripción: Crea una nueva especialidad

Roles Requeridos: `Admin`

Body:

```
{
  "name": "string",
  "description": "string"
}
```

Respuesta (201):

```
{
  "id": 1,
  "name": "Cardiología"
}
```

6. Actualizar Especialidad (Admin)

```
PUT /api/admin/specialties/:id
Authorization: Bearer <token>
```

Descripción: Actualiza una especialidad

Roles Requeridos: `Admin`

Parámetros:

- `id` - ID de la especialidad

Body:

```
{
  "name": "string",
  "description": "string"
}
```

Respuesta (200):

```
{
  "id": 1,
  "name": "Cardiología"
}
```

---

7. Eliminar Especialidad (Admin)

```
DELETE /api/admin/specialties/:id
Authorization: Bearer <token>
```

Descripción: Elimina una especialidad

Roles Requeridos: `Admin`

Parámetros:

- `id` - ID de la especialidad

Respuesta (200):

```
{
  "message": "Specialty deleted successfully"
}
```

Horarios

1. Obtener Fechas Disponibles (Doctor, Especialidad, Ubicación)

```
POST /api/schedules/available/doctor-specialty-location
Authorization: Bearer <token>
```

Descripción: Obtiene las fechas disponibles para un doctor, especialidad y ubicación específicos

Roles Requeridos: `AdminOrPatient`

Body:

```
{
  "doctorId": 1,
  "specialtyId": 1,
  "locationId": 1
}
```

Respuesta (200):

```
{
  "dates": [
    "2025-06-15",
    "2025-06-16",
    "2025-06-17"
  ]
}
```

---

2. Obtener Fechas Disponibles (Especialidad, Ubicación)

```
POST /api/schedules/available/specialty-location
Authorization: Bearer <token>
```

Descripción: Obtiene las fechas disponibles para una especialidad y ubicación

Roles Requeridos: `AdminOrPatient`

Body:

```
{
  "specialtyId": 1,
  "locationId": 1
}
```

Respuesta (200):

```
{
  "dates": [
    "2025-06-15",
    "2025-06-16"
  ]
}
```

---

3. Obtener Horarios Disponibles (Doctor, Especialidad, Ubicación)

```
POST /api/schedules/doctor-specialty-location
Authorization: Bearer <token>
```

Descripción: Obtiene los horarios disponibles para un doctor, especialidad y ubicación en una fecha

Roles Requeridos: `AdminOrPatient`

Body:

```
{
  "doctorId": 1,
  "specialtyId": 1,
  "locationId": 1,
  "date": "2025-06-15"
}
```

Respuesta (200):

```
{
  "schedules": [
    "10:00",
    "10:30",
```

```
    "11:00",
    "11:30"
  ]
}
```

---

4. Obtener Horarios Disponibles (Especialidad, Ubicación)

```
POST /api/schedules/specialty-location
Authorization: Bearer <token>
```

Descripción: Obtiene los horarios disponibles para una especialidad y ubicación en una fecha

Roles Requeridos: `AdminOrPatient`

Body:

```
{
  "specialtyId": 1,
  "locationId": 1,
  "date": "2025-06-15"
}
```

Respuesta (200):

```
{
  "schedules": [
    "09:00",
    "09:30",
    "10:00"
  ]
}
```

Pacientes

1. Obtener Todos los Pacientes

```
GET /api/patients
Authorization: Bearer <token>
```

Descripción: Obtiene la lista de todos los pacientes

Roles Requeridos: `DoctorOrPatient`

Respuesta (200):

```
[
  {
    "id": 1,
    "firstName": "Juan",
    "lastName": "Pérez",
    "email": "juan@example.com"
  }
]
```

---

2. Obtener Paciente por dni

```
POST /api/patients/nationalId
Authorization: Bearer <token>
```

Descripción: Obtiene la información de un paciente por su dni

Roles Requeridos: `Patient`

Body:

```
{
  "nationalId": "string"
}
```

Respuesta (200):

```
{
  "id": 1,
  "nationalId": "12345678",
  "firstName": "Juan",
  "lastName": "Pérez",
  "email": "juan@example.com"
}
```

---

3. Crear Paciente

```
POST /api/patients
```

Descripción: Crea un nuevo paciente en el sistema

Body:

```
{
  "nationalId": "string",
  "firstName": "string",
  "lastName": "string",
  "email": "string",
  "phoneNumber": "string",
  "dateOfBirth": "2000-01-15",
  "healthInsuranceId": 1
}
```

Respuesta (201):

```
{
  "id": 1,
  "nationalId": "12345678",
  "firstName": "Juan",
  "lastName": "Pérez"
}
```

Seguros de Salud

1. Obtener Lista de Seguros

```
GET /api/health-insurance
```

Descripción: Obtiene la lista completa de seguros de salud disponibles

Respuesta (200):

```
[
  {
    "id": 1,
    "name": "Seguros Monterrey New York Life",
    "code": "SMNYL"
  },
  {
```

```
    "id": 2,
    "name": "IMED",
    "code": "IMED"
  }
]
```

---

2. Obtener Seguro por ID

```
GET /api/health-insurance/:id
```

Descripción: Obtiene la información de un seguro específico

Parámetros:

- `id` - ID del seguro

Respuesta (200):

```
{
  "id": 1,
  "name": "Seguros Monterrey New York Life",
  "code": "SMNYL"
}
```

---

3. Crear Seguro (Admin)

```
POST /api/admin/health-insurance
Authorization: Bearer <token>
```

Descripción: Crea un nuevo seguro de salud

Roles Requeridos: `Admin`

Body:

```
{
  "name": "string",
  "code": "string"
}
```

Respuesta (201):

```
{
  "id": 1,
  "name": "Nuevo Seguro",
  "code": "NS"
}
```

---

4. Actualizar Seguro (Admin)

```
PUT /api/admin/health-insurance/:id
Authorization: Bearer <token>
```

Descripción: Actualiza un seguro de salud

Roles Requeridos: `Admin`

Parámetros:

- `id` - ID del seguro

Body:

```
{
  "name": "string",
  "code": "string"
}
```

Respuesta (200):

```
{
  "id": 1,
  "name": "Seguro Actualizado"
}
```

---

5. Eliminar Seguro (Admin)

```
DELETE /api/admin/health-insurance/:id
Authorization: Bearer <token>
```

Descripción: Elimina un seguro de salud

Roles Requeridos: `Admin`

Parámetros:

- `id` - ID del seguro

Respuesta (200):

```
{
  "message": "Health insurance deleted successfully"
}
```

Ubicaciones

1. Obtener Ubicaciones

```
GET /api/locations
Authorization: Bearer <token>
```

Descripción: Obtiene la lista de ubicaciones

Roles Requeridos: `AdminOrPatient`

Respuesta (200):

```
[
  {
    "id": 1,
    "name": "Clínica Centro",
    "address": "Calle Principal 123",
    "city": "San José",
    "phoneNumber": "2222-3333"
  }
]
```

---

2. Obtener Ubicación por ID

```
GET /api/locations/:id
Authorization: Bearer <token>
```

Descripción: Obtiene la información de una ubicación específica

Roles Requeridos: `Patient`

Parámetros:

- `id` - ID de la ubicación

Respuesta (200):

```
{
  "id": 1,
  "name": "Clínica Centro",
  "address": "Calle Principal 123",
  "city": "San José",
  "phoneNumber": "2222-3333"
}
```

---

3. Crear Ubicación (Admin)

```
POST /api/admin/locations
Authorization: Bearer <token>
```

Descripción: Crea una nueva ubicación

Roles Requeridos: `Admin`

Body:

```
{
  "name": "string",
  "address": "string",
  "city": "string",
  "phoneNumber": "string"
}
```

Respuesta (201):

```
{
  "id": 1,
  "name": "Clínica Centro"
}
```

---

4. Actualizar Ubicación (Admin)

```
PUT /api/admin/locations/:id
Authorization: Bearer <token>
```

Descripción: Actualiza una ubicación

Roles Requeridos: `Admin`

Parámetros:

- `id` - ID de la ubicación

Body:

```
{
  "name": "string",
  "address": "string",
  "city": "string",
  "phoneNumber": "string"
}
```

Respuesta (200):

```
{
  "id": 1,
  "name": "Clínica Centro"
}
```

---

5. Eliminar Ubicación (Admin)

```
DELETE /api/admin/locations/:id
Authorization: Bearer <token>
```

Descripción: Elimina una ubicación

Roles Requeridos: `Admin`

Parámetros:

- `id` - ID de la ubicación

Respuesta (200):

```
{
  "message": "Location deleted successfully"
}
```

Administración

1. Login de Administrador

```
POST /api/admin/login
```

Descripción: Autentica un administrador y retorna un JWT

Body:

```
{
  "email": "string",
  "password": "string"
}
```

Respuesta (200):

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "admin": {
    "id": 1,
    "email": "admin@clinic.com",
    "firstName": "Admin"
  }
}
```

---

2. Crear Combinación Doctor-Especialidad-Ubicación (Admin)

```
POST /api/admin/combinations
Authorization: Bearer <token>
```

Descripción: Crea una combinación de doctor, especialidad y ubicación

Roles Requeridos: `Admin`

Body:

```
{
  "doctorId": 1,
  "specialtyId": 1,
  "locationId": 1
}
```

Respuesta (201):

```
{
  "id": 1,
  "doctorId": 1,
  "specialtyId": 1,
  "locationId": 1
}
```

---

3. Eliminar Combinación (Admin)

```
DELETE /api/admin/combinations
Authorization: Bearer <token>
```

Descripción: Elimina una combinación doctor-especialidad-ubicación

Roles Requeridos: `Admin`

Body:

```
{
  "doctorId": 1,
  "specialtyId": 1,
  "locationId": 1
}
```

Respuesta (200):

```
{
  "message": "Combination deleted successfully"
}
```

---

4. Obtener Combinaciones (Admin)

```
GET /api/admin/combinations
Authorization: Bearer <token>
```

Descripción: Obtiene todas las combinaciones doctor-especialidad-ubicación

Roles Requeridos: `Admin`

Respuesta (200):

```
[
  {
    "id": 1,
    "doctorId": 1,
    "doctorName": "Dr. Carlos López",
    "specialtyId": 1,
    "specialtyName": "Cardiología",
    "locationId": 1,
    "locationName": "Clínica Centro"
  }
]
```

---

5. Crear Horario (Admin)

```
POST /api/admin/schedules
Authorization: Bearer <token>
```

Descripción: Crea un nuevo horario para un doctor

Roles Requeridos: `Admin`

Body:

```
{
  "doctorId": 1,
  "date": "2025-06-15",
  "startTime": "09:00",
  "endTime": "17:00",
  "duration": 30
}
```

Respuesta (201):

```
{
  "id": 1,
  "doctorId": 1,
  "date": "2025-06-15"
}
```

---

6. Reemplazar Horarios (Admin)

```
POST /api/admin/schedules/replace
Authorization: Bearer <token>
```

Descripción: Reemplaza los horarios de un doctor con nuevos horarios

Roles Requeridos: `Admin`

Body:

```
{
  "doctorId": 1,
  "schedules": [
    {
      "date": "2025-06-15",
      "startTime": "09:00",
      "endTime": "17:00"
    }
  ]
}
```

Respuesta (200):

```
{
  "message": "Schedules replaced successfully"
}
```

---

7. Obtener Horarios de un Doctor (Admin)

```
POST /api/admin/schedules/doctor
Authorization: Bearer <token>
```

Descripción: Obtiene todos los horarios de un doctor específico

Roles Requeridos: `Admin`

Body:

```
{
  "doctorId": 1
}
```

Respuesta (200):

```
[
  {
    "id": 1,
    "date": "2025-06-15",
    "startTime": "09:00",
    "endTime": "17:00"
  }
]
```

---

8. Actualizar Horario (Admin)

```
PUT /api/admin/schedules
Authorization: Bearer <token>
```

Descripción: Actualiza un horario existente

Roles Requeridos: `Admin`

Body:

```
{
  "scheduleId": 1,
  "date": "2025-06-15",
  "startTime": "09:00",
  "endTime": "17:00"
}
```

Respuesta (200):

```
{
  "id": 1,
  "date": "2025-06-15"
}
```

Emails

1. Enviar Email

```
POST /api/send-email
Authorization: Bearer <token>
```

Descripción: Envía un email a través del sistema

Roles Requeridos: `Patient`

Body:

```
{
  "to": "string",
  "subject": "string",
  "message": "string"
}
```

Respuesta (200):

```
{
  "message": "Email sent successfully"
}
```

Estudios Médicos

1. Subir Estudio

```
POST /api/studies/upload
Authorization: Bearer <token>
Content-Type: multipart/form-data
```

Descripción: Carga un archivo de estudio médico

Roles Requeridos: `Doctor`

Body (FormData):

- `file` - Archivo del estudio
- `patientId` - ID del paciente
- `description` - Descripción del estudio (opcional)

Respuesta (201):

```
{
  "id": 1,
  "fileName": "estudio.pdf",
  "patientId": 1,
  "doctorId": 1,
  "uploadedAt": "2025-06-15T10:00:00"
}
```

---

2. Obtener Estudios del Paciente

```
GET /api/studies/patient/:patientId
Authorization: Bearer <token>
```

Descripción: Obtiene todos los estudios de un paciente

Roles Requeridos: `Patient`

Parámetros:

- `patientId` - ID del paciente

Respuesta (200):

```
[
  {
    "id": 1,
    "fileName": "estudio.pdf",
    "description": "Resonancia magnética",
    "uploadedAt": "2025-06-15T10:00:00"
  }
]
```

---

3. Obtener Estudios del Doctor

```
GET /api/studies/doctor/:doctorId
Authorization: Bearer <token>
```

Descripción: Obtiene todos los estudios subidos por un doctor

Roles Requeridos: `Doctor`

Parámetros:

- `doctorId` - ID del doctor

Respuesta (200):

```
[
  {
    "id": 1,
    "fileName": "estudio.pdf",
    "patientId": 1,
    "patientName": "Juan Pérez",
    "uploadedAt": "2025-06-15T10:00:00"
  }
]
```

---

4. Descargar Estudio

```
GET /api/studies/download/:id
Authorization: Bearer <token>
```

Descripción: Descarga un archivo de estudio

Roles Requeridos: `DoctorOrPatient`

Parámetros:

- `id` - ID del estudio

Respuesta: Archivo binario (PDF, imagen, etc.)

---

5. Eliminar Estudio

```
DELETE /api/studies/:id
Authorization: Bearer <token>
```

Descripción: Elimina un estudio del sistema

Roles Requeridos: `Doctor`

Parámetros:

- `id` - ID del estudio

Respuesta (200):

```
{
  "message": "Study deleted successfully"
}
```

Códigos de Respuesta HTTP

Código	Significado
`200`	OK - Solicitud exitosa
`201`	Created - Recurso creado exitosamente
`400`	Bad Request - Datos inválidos
`401`	Unauthorized - No autenticado
`403`	Forbidden - No autorizado
`404`	Not Found - Recurso no encontrado
`500`	Internal Server Error - Error del servidor

Manejo de Errores

Las respuestas de error típicamente tienen el siguiente formato:

```
{
  "error": "Descripción del error",
  "message": "Mensaje detallado",
  "statusCode": 400
}
```

Notas Importantes

- Todos los endpoints requieren Content-Type: `application/json` excepto los de subida de archivos
- Los tokens JWT deben ser incluidos en el header `Authorization: Bearer <token>`

- Algunos endpoints tienen restricción de roles específicos
- Las fechas deben estar en formato ISO 8601: `YYYY-MM-DD` o `YYYY-MM-DDTHH:mm:ss`
- Los tiempos deben estar en formato 24 horas: `HH:mm`
- La API ejecuta automáticamente tareas de cancelación y recordatorios de citas

---

Variables de Entorno Requeridas

```
PORT=3000
JWT_SECRET=tu_secreto_jwt_aqui
DATABASE_URL=tu_url_de_base_de_datos
NODE_ENV=development
```

Última actualización: 2026-05-05